SUMMER INTERNSHIP / PROJECT REPORT

PERSONAL TO-DO LIST APPLICATION

Full Stack Application Development with ReactJS / MERN Stack

Submitted by: Albi Xavier
Institution: Sahrdaya College of Engineering and Technology Kodakara
Course / Batch: B tech CSE , S3
Submitted to: Dr.Anil Antony
Submission Date: 24/10/2026

EXECUTIVE SUMMARY

The Personal To-Do List is a web application developed to help a user record and manage everyday tasks. The application allows tasks to be created with a title and description, with optional due date and priority. Users can mark tasks as completed, edit a task title, delete tasks, and filter the list by All, Pending, or Completed. The application uses React for the user interface, Node.js and Express for the API, and MongoDB Atlas for persistent storage, with Mongoose used to define and access task records. Axios is used by the frontend to make HTTP requests to the API.

The source code is maintained in Git and published on GitHub. The frontend is deployed on Vercel and the backend API is deployed on Render. During functional checks reported during development, the main task-management flows and persistence after refresh were tested successfully. The application is intentionally implemented without user authentication; therefore, it is a basic personal-task demonstration rather than a multi-user private task service.

INTRODUCTION

People often need a simple way to keep track of small activities, deadlines, and priorities. When tasks are recorded in separate notes or remembered mentally, it can be difficult to review outstanding work or identify completed items. The Personal To-Do List project addresses this everyday problem by providing a single web interface for creating, viewing, updating, completing, filtering, and removing tasks.

The project was built as a full-stack JavaScript application using the MERN approach. React runs in the browser and communicates with an Express API over HTTP. The API uses Mongoose to read and write task documents in MongoDB Atlas. The browser does not connect directly to MongoDB. The deployed frontend and backend are available through the links in the appendix.

OBJECTIVES

1. Build a functional web-based to-do list interface using React.
2. Create a Node.js and Express API for task operations.
3. Store task data persistently in MongoDB Atlas using Mongoose.
4. Implement task creation, completion, title editing, deletion, and status filtering.
5. Support optional due dates and task priorities.
6. Use Git and GitHub to track and publish source code.
7. Build and deploy the frontend and backend as separate services.
8. Practice environment-variable configuration and production API integration.

SCOPE AND DELIVERABLES

Scope included

- Create tasks with a required title and optional description, due date, and priority.
- Display tasks and summary counts.
- Mark tasks as completed or pending.
- Edit the task title.

- Delete tasks.
- Filter tasks by All, Pending, and Completed.
- Persist task records in MongoDB so they remain after a page refresh.
- Provide a REST-style API under /api/tasks and a health-check endpoint.
- Publish the source code on GitHub and deploy the frontend and API.

Out of scope / current limitations

- No sign-up, login, or authentication is implemented.
- Tasks are not separated by individual user accounts; anyone who can access the public app may interact with the shared task collection.
- The current edit interface supports changing the title; a full edit form for every task field is not included.
- Automated unit or end-to-end test suites are not documented as implemented; the checks described here are manual functional checks.
- Custom domain configuration and advanced monitoring are not included.

Deliverables

- React frontend source in the client folder.
- Express API, routes, and Mongoose model in the server folder.
- MongoDB Atlas database connection configured through environment variables.
- GitHub repository and deployed frontend/backend URLs.
- This project report and technical documentation.

METHODOLOGY

The project was developed incrementally. The frontend and backend were first set up as separate applications. The backend was configured with Express, CORS, dotenv, and Mongoose; a task schema was created and CRUD routes were implemented. The frontend was then connected to the API with Axios. After the core flows were checked locally, the source was committed to Git and pushed to GitHub. The API was deployed to Render and the React production build was deployed to Vercel using an environment variable for the production API URL.

Technology stack

Layer	Technology	Purpose
Frontend	React, Vite, CSS	User interface and production build
HTTP client	Axios	Requests from frontend to backend API
Backend	Node.js, Express	HTTP API and task operations
Database	MongoDB Atlas	Persistent storage of task documents
ODM	Mongoose	Task schema and database interaction

Configuration	dotenv / environment variables	Local and hosted configuration
Version control	Git, GitHub	Source tracking and remote repository
Hosting	Vercel, Render	Frontend and backend deployment

PROJECT ACTIVITIES

Project setup: Created a project with separate client and server folders. Installed the required frontend and backend packages.

Database and model: Configured MongoDB Atlas and created a Mongoose Task model with title, description, dueDate, priority, completed, and automatic timestamps.

API implementation: Added a health-check endpoint and task endpoints for listing, creating, updating, and deleting tasks.

Frontend integration: Connected React to the API using Axios and configured the API URL to support local development and hosted deployment.

Functional checks: Checked adding tasks, optional task details, completion, title editing, deletion, filters, and persistence after refresh.

Version control: Initialized Git, added ignore rules for environment files and generated/dependency folders, committed the source, and pushed it to GitHub.

Deployment: Deployed the backend API on Render and the React frontend on Vercel; confirmed the health-check response and reported that the live app's core flows worked.

RESULTS & FINDINGS

The project reached a working deployed state. The live React application communicates with the hosted Express API, and task records persist in MongoDB Atlas. The production frontend build completed successfully during the final preparation stage. The developer also confirmed that the deployed application's task-management flows worked.

Test case	Expected result	Reported result
Add task with title and description	Task is saved and shown in list	Passed
Add task with due date and priority	Optional details are saved	Passed
Mark task complete	Completion state changes	Passed
Edit task title	Updated title is displayed	Passed
Delete task	Task is removed from list	Passed
Filter All / Pending / Completed	List shows matching tasks	Passed

Refresh page	Saved tasks remain available	Passed
API health check	JSON confirms API is running	Passed
Production frontend build	Vite build completes	Passed

These results are based on manual checks performed during development and user confirmation, not on a formal automated test suite. No performance, load, penetration, or accessibility audit is claimed.

CONCLUSION

The Personal To-Do List project demonstrates a complete beginner-level MERN application: a React interface, a Node.js/Express API, and a MongoDB-backed data model. The main task-management operations are available and the application has been deployed with separate frontend and backend services. The work provided practical experience with REST endpoints, database schemas, asynchronous API requests, environment variables, Git/GitHub, and deployment.

Future improvements could include authentication and per-user task ownership, complete editing of all task fields, stronger server-side validation and centralized error handling, automated tests, pagination for large task lists, accessibility and responsive-layout review, and more restrictive CORS configuration. Because authentication is not implemented, the deployed app should be treated as a learning/demo project and should not be used to store private task information.

APPENDIX

A. Project links

Live frontend: https://personal-todo-list-nine.vercel.app/

Backend health endpoint: https://personal-todo-list-api.onrender.com/api/health

GitHub repository: https://github.com/Albisahrdaya/personal-todo-list

B. Project structure

- client/ — React and Vite frontend.
- client/src/App.jsx — main task UI and API calls.
- client/src/App.css and client/src/index.css — styling.
- server/server.js — Express application, MongoDB connection, and API registration.
- server/models/Task.js — Mongoose task schema.
- server/routes/tasks.js — CRUD routes for tasks.
- .gitignore — excludes dependencies, environment files, and frontend build output.

C. API endpoint summary

Method	Endpoint	Purpose
GET	/api/health	Health check
GET	/api/tasks	Retrieve tasks
POST	/api/tasks	Create a task

PUT	/api/tasks/:id	Update a task
DELETE	/api/tasks/:id	Delete a task

D. Task data model

- title: required string, trimmed, maximum length 100 characters.
- description: optional trimmed string; defaults to an empty string.
- dueDate: optional date; defaults to null.
- priority: Low, Medium, or High; defaults to Medium.
- completed: Boolean; defaults to false.
- createdAt and updatedAt: maintained by Mongoose timestamps.

E. Screenshot placeholders

Insert actual screenshots of the deployed application in the final copy. Suggested evidence:

- [Insert Screenshot 1: Live dashboard and task summary]
- [Insert Screenshot 2: Add task form]
- [Insert Screenshot 3: Task list and filters]
- [Insert Screenshot 4: Completed task state]
- [Optional: Insert MongoDB Atlas task document view; ensure credentials are not visible]

F. Declaration and review

Before submission, replace all bracketed placeholders, insert genuine screenshots, verify the links, and ensure the report matches the actual work and your institution's formatting requirements.